



SE507 - SOFTWARE TESTING AND EVALUATION

A Comprehensive Testing Strategy for a Library Management System

Team Members:

Tishko Salah (Test Manager)
Sara Zuhair (Code specifier)
Yassin Hussein (Code reviewer)
Ravyar Barzan (Document inspector)

Module Leader: **Dr. Hossein Hassani**

June 15, 2025

University of Kurdistan Hewlêr
Department of Computer Science and Engineering
Erbil, Kurdistan Region, Iraq

Contents

1	Background	5
1.1	System Under Test (SUT) Overview	5
1.2	System Architecture	5
1.3	User Roles and Functionalities	6
1.3.1	Librarian (Admin)	6
1.3.2	Member	6
1.4	Data Model	7
2	Test Management	8
2.1	Introduction	8
2.2	Test Strategy	8
2.2.1	Overall Approach	8
2.2.2	Test Levels	8
2.3	Test Scope	9
2.3.1	In Scope	9
2.3.2	Out of Scope	9
2.4	Team Roles and Responsibilities	10
2.5	Test Environment and Tools	10
2.6	Test Schedule	11
2.7	Deliverables	11
2.8	Risk Management	11
2.9	Entry and Exit Criteria	12
2.9.1	Entry Criteria	12
2.9.2	Exit Criteria	12
3	Testing Process	14
3.1	Introduction	14
3.2	Test Environment and Configuration	16
3.3	Testing Levels and Automation Frameworks	16
3.3.1	Unit Testing	16
3.3.2	Integration Testing	17
3.3.3	System (End-to-End) Testing	18
3.4	Test Execution and Reporting	19
3.5	Defect Management	19
4	Functional Testing	21
4.1	Authentication and Authorization Testing	21
4.2	Entity Management (CRUD Operations) Testing	25
4.2.1	Book Management	26
4.2.2	Author Management	26
4.2.3	Genre Management	26

4.2.4	User Management	26
4.2.5	Borrowal Management	26
4.2.6	Review Management	26
4.3	Specific Feature Testing	33
4.3.1	Dashboard (Librarian)	33
4.3.2	Book Management (Detailed)	33
4.3.3	Borrowal Management (Detailed)	34
4.3.4	User Management (by Librarian) (Detailed)	34
4.3.5	Review Management (Detailed)	34
4.4	Use Case Testing	38
4.4.1	UC-002: Add New Book (Librarian)	38
4.4.2	UC-003: Borrow a Book (Member)	38
4.4.3	UC-005: View Borrowal History (Member)	38
4.5	State Transition Testing	42
4.5.1	Borrowal Record Status	42
4.5.2	Book Availability Status	42
4.5.3	User Session State	42
5	API Testing	46
5.1	Introduction to API Testing	46
5.2	Test Design and Strategy	46
5.2.1	Test Case Derivation	46
5.2.2	Manual Testing with Postman	47
5.3	Automated API Testing with Jest and Supertest	47
5.3.1	Framework and Setup	47
5.3.2	Example Test Implementation	48
5.4	Test Execution and Key Findings	48
5.4.1	Discrepancy in Access Control	48

List of Figures

1.1	High-Level System Architecture.	6
1.2	Conceptual Entity-Relationship Diagram (ERD).	7
2.1	Project Testing Schedule Gantt Chart	11
3.1	High-Level Automated Testing Process Flowchart	15
3.2	Docker-Based Test Environment Architecture	16
3.3	Example of an HTML Test Report from Jest	19
4.1	Librarian Dashboard Overview	33
4.2	Member's Borrowal History View	38
5.1	Manual API Testing using Postman	47

List of Tables

2.1	Team Roles and Primary Responsibilities	10
2.2	Testing Tools and Frameworks	10
2.3	Risk Assessment and Mitigation Plan	12
4.1	Authentication and Authorization Test Cases	22
4.2	Entity Management Test Cases	26
4.3	Specific Feature Test Cases	34
4.4	Use Case Test Cases	39
4.5	State Transition Test Cases	43

Background

This chapter provides a comprehensive overview of the Library Management System, the System Under Test (SUT). It details the system's purpose, architecture, user roles, core functionalities, and underlying data model. This context is essential for understanding the scope, strategy, and execution of the testing activities detailed in subsequent chapters.

1.1 System Under Test (SUT) Overview

The Library Management System is a full-stack web application designed to modernize and streamline library operations **srs**. Its primary purpose is to provide a digital platform for managing library resources, including books, authors, and genres, while also managing user accounts for both librarians and library members **srs**. The system aims to replace or supplement traditional, manual library management methods with an efficient, accessible, and user-friendly web-based solution **srs**.

The project's scope encompasses a complete, self-contained system featuring a backend API, a database, and a frontend client application **srs**. Key capabilities include:

- **User Authentication:** Secure user registration, login, and logout functionalities with role-based access control for two distinct user types: Librarians and Members **srs**.
- **Inventory Management:** Comprehensive CRUD (Create, Read, Update, Delete) operations for the core library entities: books, authors, and genres **srs**.
- **Borrowal Tracking:** Management of book borrowals and returns, allowing librarians to oversee lending activities and members to view their history **srs**.
- **User Administration:** Tools for librarians to manage all user accounts within the system **srs**.
- **Review System:** A newly introduced feature allowing members to submit reviews for books, which can then be managed by librarians **srs**.

1.2 System Architecture

The system is built on a modern, multi-tier architecture, which separates concerns between the client, server, and database. This design promotes modularity, scalability, and maintainability.

- **Frontend (Client-Side):** The user interface is a single-page application (SPA) developed using **React** and the **Material-UI (MUI)** component library. It is responsible for rendering the UI and interacting with the backend via API calls. It runs in any modern web browser **srs**.

- **Backend (Server-Side):** The backend is a RESTful API built with **Node.js** and the **Express.js** framework. It handles all business logic, data processing, and user authentication using Passport.js **srs**.
- **Database:** Data persistence is managed by a **MongoDB** database, a NoSQL database that stores data in flexible, JSON-like documents. The backend interacts with MongoDB via the Mongoose ODM library **srs**.

The entire application is containerized using **Docker**, ensuring consistent development, testing, and deployment environments **srs**. Figure 1.1 illustrates the high-level architecture.

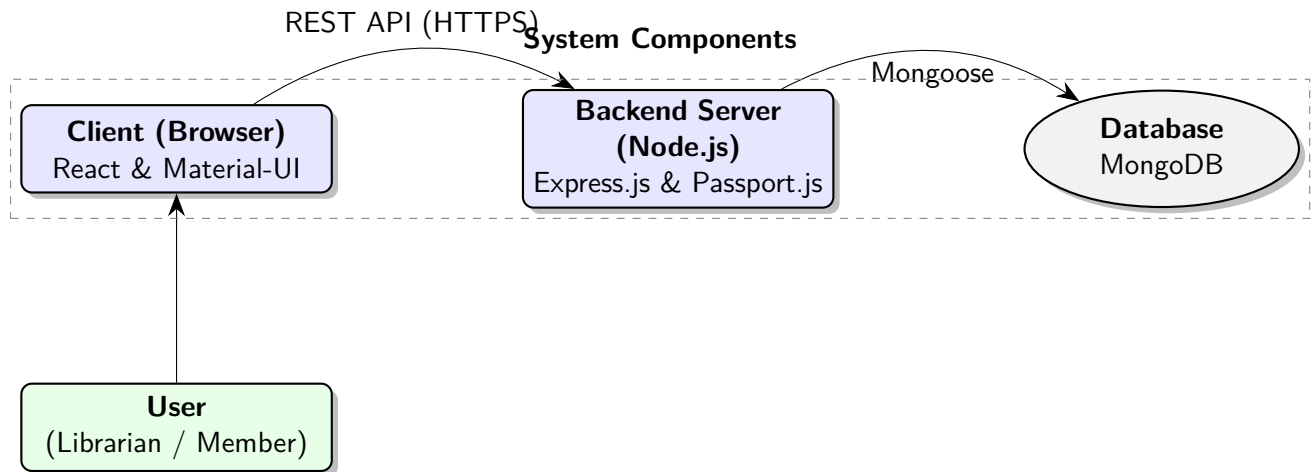


Figure 1.1: High-Level System Architecture.

1.3 User Roles and Functionalities

The system defines two distinct user roles, each with a specific set of permissions and responsibilities as outlined in the SRS **srs**.

1.3.1 Librarian (Admin)

Librarians are the administrators of the system with full control over all its aspects. Their primary responsibilities include:

- Managing the library's entire inventory, including adding, updating, and deleting books, authors, and genres **srs**.
- Overseeing all user accounts, which involves creating new accounts for both members and other librarians, as well as updating and deleting existing accounts **srs**.
- Managing the complete lifecycle of book borrowals, from creation to marking returns **srs**.
- Viewing system-wide statistics and data on a dedicated dashboard **srs**.
- Moderating book reviews submitted by members **srs**.

1.3.2 Member

Members are the standard users of the library. Their access is generally limited to viewing data and managing their own activities. Their functions include:

- Browse the library's catalog of books, authors, and genres **srs**.

- Requesting to borrow available books **srs**.
- Viewing their personal borrowing history **srs**.
- Submitting, viewing, and managing their own reviews for books **srs**.

1.4 Data Model

The system's data is structured around several core entities, whose relationships define the application's functionality. The main entities are **User**, **Book**, **Author**, **Genre**, **Borrowal**, and **Review srs**. The relationships between these entities are a key focus of integration testing to ensure data integrity. For instance, a *Borrowal* record links a *User* to a *Book*, and a *Book* can have one *Author* and one *Genre srs*.

Figure 1.2 provides a conceptual Entity-Relationship Diagram (ERD) illustrating these connections.

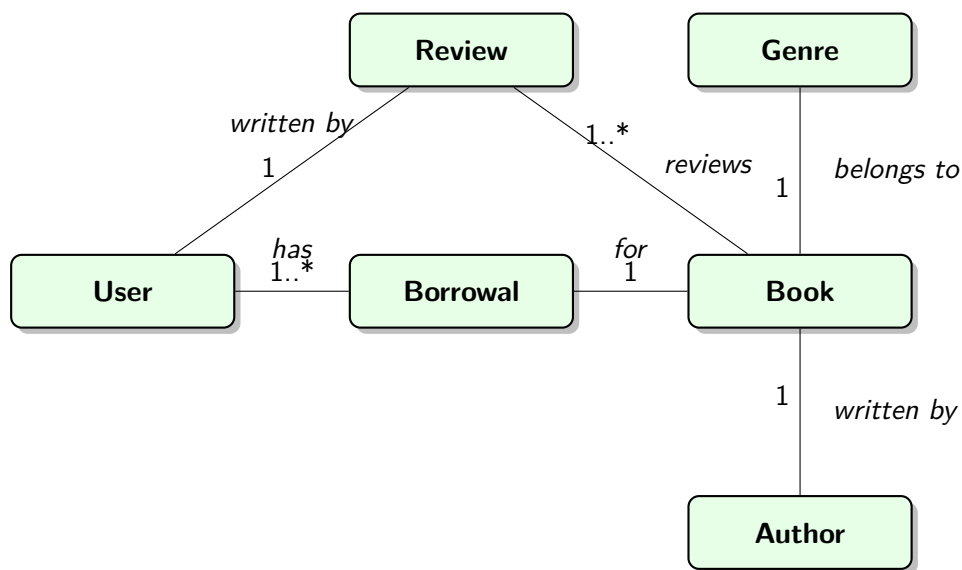


Figure 1.2: Conceptual Entity-Relationship Diagram (ERD).

Test Management

2.1 Introduction

This chapter outlines the comprehensive management plan for the testing of the Library Management System. The plan is designed to ensure that all software components are rigorously tested, meet the requirements specified in the Software Requirements Specification (SRS), and achieve the highest standards of quality. This document details the test strategy, scope, team roles, environment, schedule, deliverables, risk management, and success criteria for the project.

2.2 Test Strategy

Our test strategy employs a multi-layered approach, integrating various testing levels to ensure comprehensive coverage and early defect detection. The strategy is aligned with the V-Model, ensuring that testing activities are planned in parallel with development phases.

2.2.1 Overall Approach

The testing process is structured across multiple levels:

- **Static Testing:** Performed without executing the code, focusing on reviews and analysis of documentation and source code to find defects early.
- **Unit Testing:** Developers will test individual components (functions, classes) in isolation to verify their correctness.
- **Integration Testing:** Focuses on testing the interfaces and interactions between integrated components, such as the communication between the frontend client and the backend API.
- **System & E2E Testing:** The complete, integrated system is tested from end-to-end to validate that it meets all specified requirements from a user's perspective.
- **Non-Functional Testing:** Ensures the system meets requirements for performance, security, usability, and compatibility.
- **Regression Testing:** Conducted after modifications to ensure that new changes have not introduced new defects into existing functionality.

2.2.2 Test Levels

- **Unit Testing:** Implemented using **Jest**. For the backend, this involves testing Node.js controllers, models, and utility functions. For the frontend, this includes testing React components, hooks, and utility functions using Jest and the React Testing Library.

- **API & Integration Testing:** Backend API endpoints are tested using **Jest** and **Supertest** to verify CRUD operations, authentication flows, and business logic. This also covers the integration points between different services.
- **End-to-End (E2E) Testing:** Real user workflows are simulated using **Cypress**. These tests cover complete user journeys, such as logging in, searching for a book, borrowing it, and logging out.
- **Non-Functional Testing:**
 - **Performance Testing:** **K6** is used to script and execute load tests against the API to measure response times, throughput, and stability under load.
 - **Security Testing:** Manual and automated checks are performed to identify common vulnerabilities (e.g., SQL injection, XSS, insecure direct object references). Cypress tests are used to check for authorization flaws.
 - **Usability & Compatibility Testing:** Manual testing and Cypress scripts ensure the application is user-friendly and renders correctly across major web browsers (Chrome, Firefox, Safari).

2.3 Test Scope

2.3.1 In Scope

The following features and functionalities, as defined in the SRS, are within the scope of testing:

- User Authentication (Login, Logout, Role-based Access Control)
- Librarian Dashboard and statistical widgets
- Full CRUD (Create, Read, Update, Delete) operations for Books, Authors, and Genres
- Borrowal Management (borrowing, returning, and tracking books)
- User Management (Librarian managing member accounts)
- Book Review and Rating system
- API endpoint functionality and security
- Database referential integrity

2.3.2 Out of Scope

The following are considered out of scope for testing:

- Testing of third-party libraries and frameworks (e.g., React, Express, Mongoose).
- Performance of the underlying hardware, network infrastructure, or MongoDB database engine itself.
- Exhaustive testing of every possible combination of user inputs.
- Usability testing on mobile operating systems (unless specified).

2.4 Team Roles and Responsibilities

The testing process involves the entire team, with specific roles and responsibilities as defined in the project's task distribution plan.

Table 2.1: Team Roles and Primary Responsibilities

Member	Role	Primary Testing Responsibilities
Tishko Salah	Test Manager	Oversees the entire testing process, manages the test plan and schedule, coordinates team efforts, and is the final authority on test completion.
Sara Zuhair	Code Specifier	Focuses on non-coder testing activities, including functional validation against SRS, documentation review, and usability testing. Leads the execution of user-centric test cases.
Yassin Hussein	Code Reviewer	Leads technical testing activities, including unit testing, API testing, and code reviews. Ensures code quality, performance, and adherence to standards.
Ravyar Barzan	Document Inspector	Leads the review and inspection of all project documentation (SRS, Test Plan, Test Cases, Final Report) for clarity, consistency, and completeness.

2.5 Test Environment and Tools

All testing is conducted within containerized environments managed by Docker to ensure consistency and reproducibility.

- **Operating Systems:** Windows, macOS, and Linux (for development and execution).
- **Test Environments:** Defined in `docker-compose.dev.yml`, `docker-compose.test.yml`, and `docker-compose.prod.yml` to simulate development, testing, and production setups.
- **Version Control:** Git, with GitHub for repository hosting and collaboration.

Table 2.2: Testing Tools and Frameworks

Category	Tool/Framework	Purpose
Backend Testing	Node.js, Jest, Supertest	Unit and integration testing of API endpoints and services.
Frontend Testing	Jest, React Testing Library	Unit and component testing of React components and logic.
E2E Testing	Cypress	Automated end-to-end testing of user workflows in a browser.
Performance	K6, Docker	Load testing the backend API to measure performance metrics.
Code Quality	ESLint, Prettier	Static code analysis to enforce coding standards and find bugs.
Documentation	LaTeX, Draw.io	Creating project reports, diagrams, and test artifacts.

2.6 Test Schedule

The project timeline spans approximately 14 weeks, from early March to mid-June 2025. The schedule is visualized in the Gantt chart below (Figure 2.1).

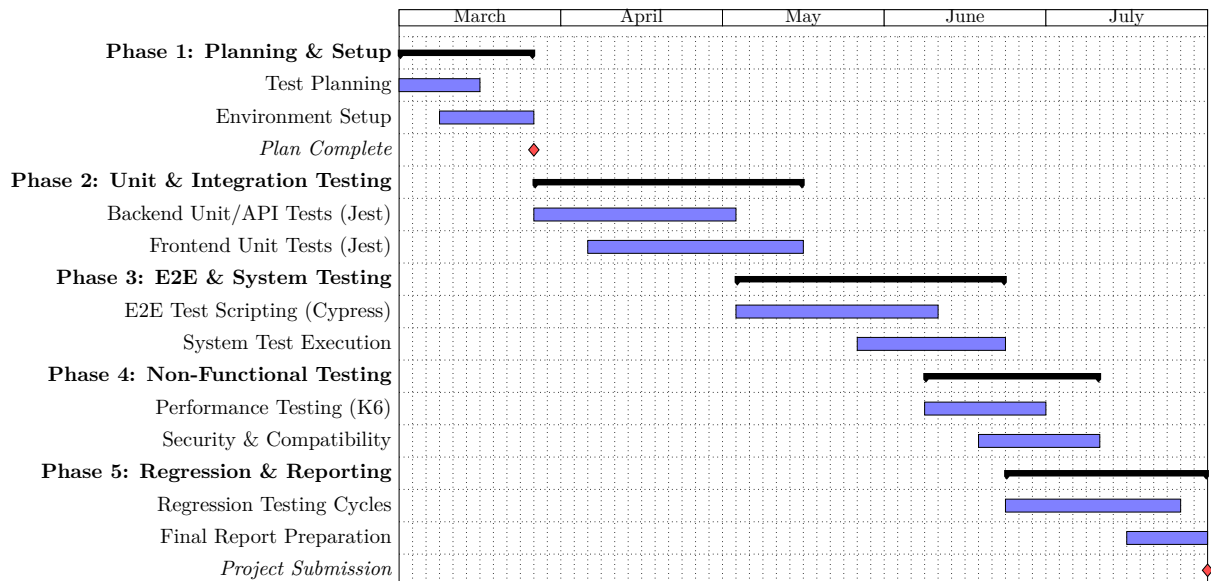


Figure 2.1: Project Testing Schedule Gantt Chart

2.7 Deliverables

The following artifacts will be produced as part of the testing process:

- **Test Plan:** This comprehensive document (of which this chapter is a part).
- **Test Case Documentation:** Detailed test cases for all levels of testing, located in `/docs/Report/test_artifacts/test_cases/`.
- **Test Scripts:** All automated test scripts written using Jest, Cypress, and K6.
- **Defect Reports:** Bugs and issues will be tracked, prioritized, and managed. (A system like GitHub Issues will be used).
- **Test Execution Reports:** Automated reports generated by Jest (`/server/test-report/report.html`) and Cypress (videos/screenshots) after test runs.
- **Final Test Summary Report:** The complete final report detailing all testing activities, results, and conclusions.

2.8 Risk Management

Potential risks that could impact the testing schedule and quality are identified and managed proactively.

Table 2.3: Risk Assessment and Mitigation Plan

Risk Description	Likelihood	Impact	Mitigation Strategy
Delays in development phases impact testing start dates.	Medium	High	Maintain close communication between developers and testers. Adapt the test plan and prioritize testing of critical features.
Test environment is unstable or differs from production.	Low	High	Use Docker to ensure consistent environments. Perform smoke tests on deployment to validate environment stability.
Critical defects are discovered late in the project timeline.	Medium	High	Emphasize early testing (unit, integration). Prioritize test cases based on risk and critical user workflows.
Team member unavailability due to unforeseen circumstances.	Low	Medium	Ensure knowledge is shared across the team. Cross-train members on critical testing tasks and tools.
Underestimation of time required for test case creation or execution.	Medium	Medium	Base estimates on feature complexity. Regularly review progress and re-allocate resources as needed.

2.9 Entry and Exit Criteria

Clear criteria define the start and end points for the formal testing process.

2.9.1 Entry Criteria

Testing on a feature or build can begin when:

- The SRS and relevant design documents are complete and approved.
- The code has been developed, peer-reviewed, and successfully built.
- All developer-written unit tests for the feature are passing.
- The required test environment and test data are available and stable.
- Test cases have been written, reviewed, and approved by the test manager.

2.9.2 Exit Criteria

The overall testing phase will be considered complete when:

- All planned test cases have been executed.
- A pass rate of 100% for all critical and major test cases is achieved.
- A pass rate of over 95% for minor test cases is achieved.
- No outstanding Severity-1 (blocker) or Severity-2 (critical) defects remain open.

- All major functional and non-functional requirements are verified and met.
- The final test report is completed and approved by all team members.

Testing Process

3.1 Introduction

The testing process for the Library Management System is a structured and multi-layered approach designed to ensure the delivery of a high-quality, reliable, and robust application. Its primary goal is to systematically verify and validate that the system meets the functional and non-functional requirements detailed in the Software Requirements Specification (SRS). This process is integral to mitigating risks, identifying defects early in the development lifecycle, and ultimately ensuring user satisfaction.

Our methodology emphasizes automation to facilitate a continuous integration and testing pipeline. By leveraging containerization with Docker, we establish consistent and isolated environments for development, testing, and production. This is crucial for achieving reliable and reproducible test outcomes, eliminating the "it works on my machine" problem. This chapter details the entire process, from the setup of the test environment to the execution of various testing levels and the management of defects. A visual representation of our high-level automated testing process is shown in Figure 3.1.

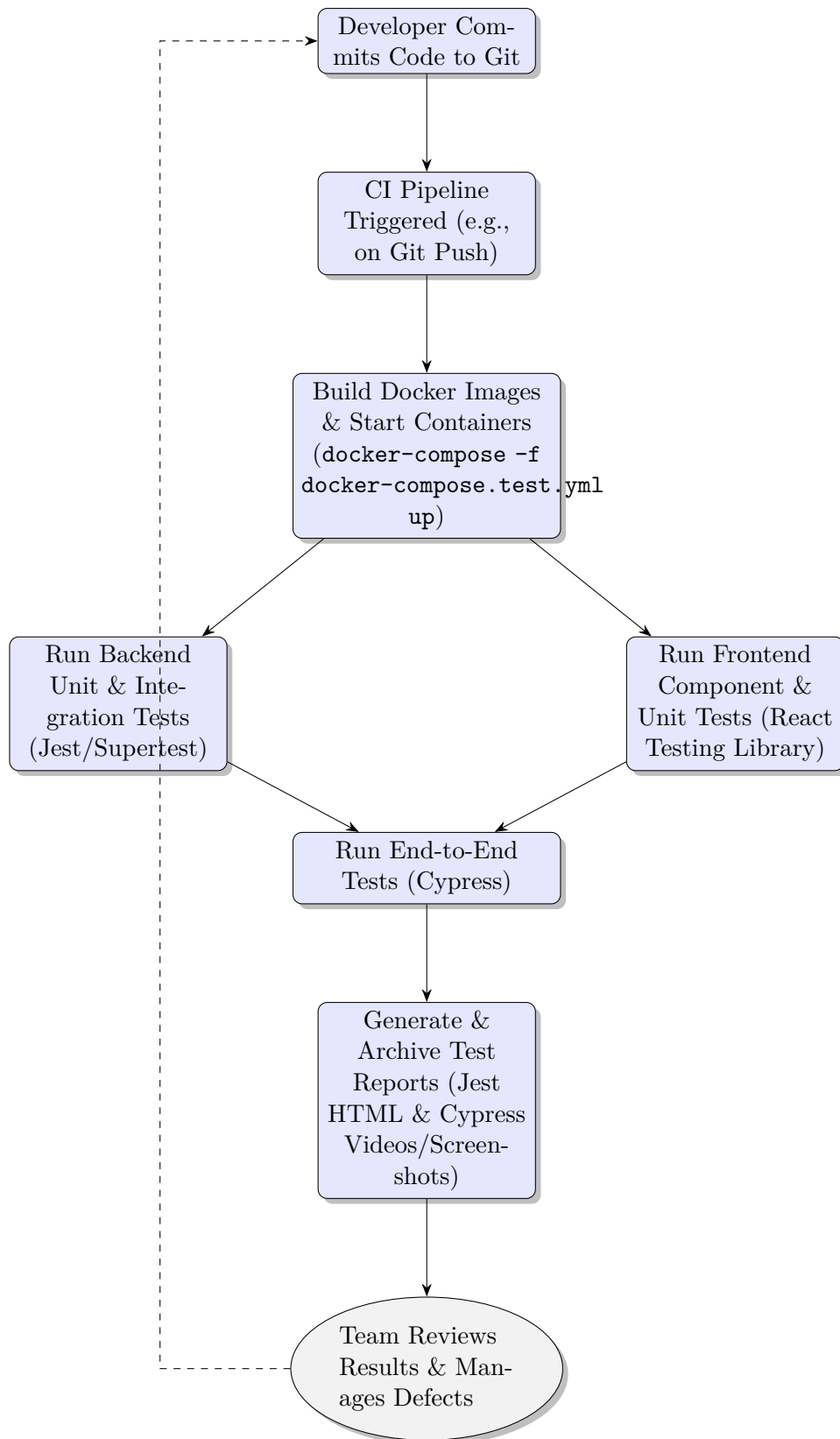


Figure 3.1: High-Level Automated Testing Process Flowchart

3.2 Test Environment and Configuration

A consistent and reproducible test environment is the cornerstone of our testing strategy. We use Docker and Docker Compose to define and manage the application stack, ensuring that tests are executed in an environment that mirrors production as closely as possible. This minimizes environment-specific defects and guarantees that a test that passes on one developer's machine will also pass in the CI pipeline.

The entire test environment is orchestrated by the `docker-compose.test.yml` file and configured via the `.env.test` file. This setup allows any team member to spin up the full test stack with a single command (`docker-compose -f docker-compose.test.yml up -build`). The services defined are:

- **server-test**: The Node.js backend application. For API and integration tests, it is configured via `.env.test` to connect to an in-memory MongoDB server. This provides significant advantages: test isolation (no data leakage between tests), speed (no disk I/O), and simplicity (no need for manual database cleanup).
- **mongo-test**: The MongoDB database. While most automated tests use the in-memory version, this service is available for specific tests that may require validation against a persistent database instance.
- **client-test**: The React frontend application, serving the user interface against which end-to-end tests are run.
- **cypress-e2e**: A dedicated container that runs the Cypress test runner for executing end-to-end tests against the client and server. It is configured to communicate with the other services within the Docker network.

This containerized architecture, illustrated in Figure 3.2, is paramount for maintaining testing integrity across all development and deployment stages.

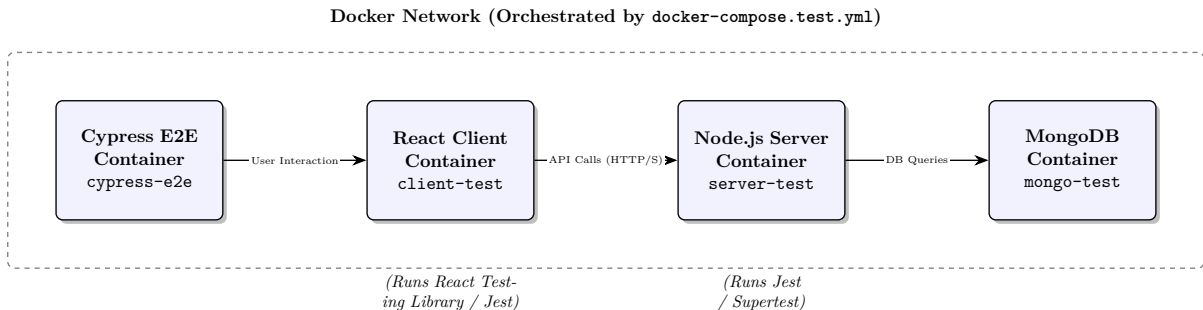


Figure 3.2: Docker-Based Test Environment Architecture

3.3 Testing Levels and Automation Frameworks

Our testing process is composed of several distinct levels, each targeting a different part of the application and utilizing specialized tools and techniques to provide comprehensive coverage.

3.3.1 Unit Testing

Unit tests form the foundation of our testing pyramid, focusing on the smallest individual components (units) of the system in isolation.

- **Backend (Server)**: We use the **Jest** framework to test individual controllers, models, and utility functions. Mocks and stubs are used extensively to isolate units from external dependencies. For example, in `bookController.test.js`, the `Book` model is mocked to

simulate database interactions, ensuring the test validates only the controller's internal logic. Similarly, tests for models themselves (e.g., `book.test.js`) verify schema validations and default values without needing a database connection.

- **Frontend (Client):** We use **React Testing Library** with Jest to test individual React components and custom hooks. Component tests verify that they render correctly given specific props and that they respond as expected to simulated user interactions. For instance, tests for `LoginForm.js` check for the presence of input fields and validate that clicking the submit button triggers the correct callback function. We also test utility functions like `formatTime.js` and `formatNumber.js` to ensure data is displayed correctly across the UI.

```
1 // From: server/__tests__/unit/controllers/bookController.test.js
2
3 const bookController = require('../../../controllers/bookController');
4 const Book = require('../../../models/book');
5
6 // Mock the Book model to isolate the controller logic
7 jest.mock('../../../models/book');
8
9 describe('Book Controller - getBookById', () => {
10   it('should return a book with status 200 if a valid ID is passed',
11     async () => {
12       const mockBook = { _id: '60c72b2f9b1d8c001f8e4c6d', title: 'Test
13       Book', author: 'Author Name' };
14       Book.findById.mockResolvedValue(mockBook);
15
16       const req = { params: { id: '60c72b2f9b1d8c001f8e4c6d' } };
17       const res = {
18         status: jest.fn().mockReturnThis(),
19         json: jest.fn()
20       };
21
22       await bookController.getBookById(req, res);
23
24       expect(res.status).toHaveBeenCalledWith(200);
25       expect(res.json).toHaveBeenCalledWith(mockBook);
26     });
27 });
```

Listing 3.1: Example of a Backend Unit Test for `bookController.js`

3.3.2 Integration Testing

Integration tests focus on verifying the interactions and data flow between different parts of the system.

- **Backend API Testing:** We use **Jest** and **Supertest** to perform integration testing on our API endpoints. These tests are critical for ensuring that controllers, models, and services work together as expected. The tests run against a live server instance connected to an in-memory MongoDB database to ensure a clean state for each test suite. We test not only basic CRUD operations but also complex business workflows. For example, `borrowal.api.test.js` tests the full lifecycle of borrowing and returning a book, while `bookReferentialIntegrity.api.test.js` ensures that a book cannot be created with a non-existent author.
- **Frontend Component Integration:** At the component level, we test how multiple components collaborate to deliver a piece of functionality. For example, on the

`AuthorPage.jsx`, we test the integration between the main data table, the `AuthorDialog.jsx` (for adding/editing), and the search/filter toolbar, ensuring that actions performed in one component correctly update the state and rendering of the others.

3.3.3 System (End-to-End) Testing

System testing validates the complete, integrated application workflow from a user's perspective. We use **Cypress** for our end-to-end (E2E) tests because of its reliability and powerful debugging features. These tests simulate real user scenarios by interacting with the application through the browser, just as a user would.

Cypress tests cover critical user journeys derived from the use cases in the SRS (Section 7), such as:

- **UC-001 (User Login):** As shown in `authentication_authorization.cy.ts`, a test logs in as an admin and verifies access to the dashboard.
- **UC-002 & UC-004 (Entity Management):** The `entity_management.cy.ts` script simulates an admin creating a new author, then a new genre, and finally a new book that uses the newly created author and genre.
- **UC-003 (Borrow a Book):** The `full_lifecycle.cy.ts` test simulates a member logging in, searching for a book, borrowing it, and then viewing it in their borrowal history.
- **Authorization:** Tests also explicitly verify access control rules, such as ensuring a non-admin user receives a "403 Forbidden" or is redirected when attempting to access administrative API endpoints or frontend routes.

To manage test data, Cypress uses fixtures (e.g., `user-data.json`) to provide consistent user credentials and other data inputs for the tests.

```
1 // From: e2e/cypress/e2e/authentication_authorization.cy.ts
2
3 describe('Authentication and Authorization', () => {
4   it('should allow an admin user to log in and see the main dashboard', ()
5     => {
6     // Visit the login page
7     cy.visit('/login');
8
9     // Use test data from a fixture file
10    cy.fixture('user-data.json').then((userData) => {
11      cy.get('input[name="email"]').type(userData.adminUser.email);
12      cy.get('input[name="password"]').type(userData.adminUser.password);
13    });
14
15    // Click the submit button
16    cy.get('button[type="submit"]').click();
17
18    // Assert that the URL has changed to the dashboard
19    cy.url().should('include', '/dashboard/app');
20
21    // Assert that a key element of the dashboard is visible to confirm
22    // success
23    cy.contains('h4', 'Welcome back').should('be.visible');
24  });
25 });
```

Listing 3.2: Example of a Cypress E2E Test for User Login

3.4 Test Execution and Reporting

Test execution is streamlined and automated through npm scripts defined in the respective `package.json` files, enabling easy execution both locally and in a CI/CD pipeline.

- **Backend Tests:** Running `npm test` in the `/server` directory executes all Jest-based unit and integration tests.
- **Frontend Tests:** Running `npm test` in the `/client` directory executes all React Testing Library component tests.
- **E2E Tests:** Running `npm run cypress:run` in the `/e2e` directory executes the full suite of Cypress end-to-end tests in headless mode.

Upon completion, the test frameworks generate detailed reports. Jest is configured with `jest-html-reporters` to produce a comprehensive HTML report (located in `server/test-report/`), which provides a shareable overview of test suite results, pass/fail status, execution times, and code coverage metrics. Code coverage is particularly important as it helps the team identify areas of the application that are not covered by tests. Cypress automatically generates video recordings of each test run and detailed screenshots upon failure, which are invaluable for quickly diagnosing the root cause of an issue.

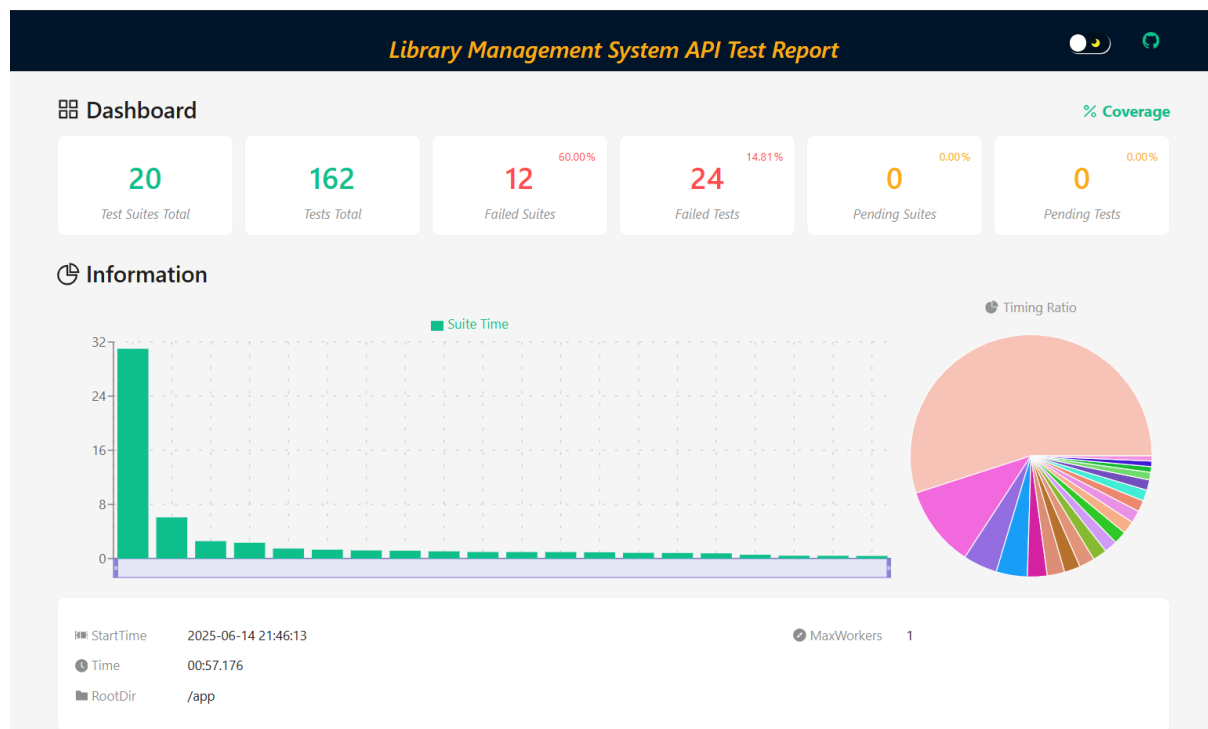


Figure 3.3: Example of an HTML Test Report from Jest

3.5 Defect Management

A formal defect management process is essential for tracking and resolving issues discovered during testing. Our process ensures that no bug is lost and that every issue is triaged, prioritized, and resolved in a timely manner. While a dedicated tool like Jira or Trello is recommended, this process can also be managed using GitHub Issues.

The defect lifecycle is as follows:

1. **Discovery and Logging:** When a test fails or a bug is found during manual testing, the tester logs a new defect. The defect report includes a clear title, a detailed description of the issue, steps to reproduce it, the expected result vs. the actual result, and any relevant artifacts like screenshots, logs, or video recordings from Cypress.
2. **Triage and Prioritization:** The Test Manager and technical lead review new defects to assess their impact and urgency. Defects are assigned a priority (e.g., Critical, High, Medium, Low) based on factors like severity, scope of impact, and risk to the project timeline.
3. **Assignment:** The defect is assigned to the appropriate developer for resolution.
4. **Resolution:** The developer fixes the bug, commits the code, and marks the defect as "Resolved." The fix should ideally include a new regression test that specifically targets the bug to prevent it from recurring.
5. **Verification:** The tester re-runs the failing test in the updated environment to verify that the fix has resolved the issue and has not introduced any new regressions. If the fix is confirmed, the defect is closed. If not, it is reopened and reassigned to the developer.

This structured workflow ensures accountability and provides clear visibility into the health of the application throughout the development process.

Functional Testing

Functional testing is a type of black-box testing that verifies the system's compliance with its specified functional requirements, as outlined in the Software Requirements Specification (SRS) document. This chapter details the functional test cases executed to ensure that each feature of the Library Management System behaves as expected from an end-user perspective, encompassing various aspects such as user interactions, data manipulations, and system responses.

The functional test cases are categorized into the following sub-sections:

- Authentication and Authorization Testing
- Entity Management (CRUD Operations) Testing
- Specific Feature Testing
- Use Case Testing
- State Transition Testing

Each sub-section includes a brief overview and references the detailed test case documents located in the `test_artifacts/test_cases/I_Functional_Testing/` directory.

4.1 Authentication and Authorization Testing

This section focuses on verifying that user authentication (login, logout, registration) and role-based access control (RBAC) mechanisms function correctly. It ensures that only authorized users can access specific features and that security measures related to user sessions are properly enforced. The test cases cover successful logins for both Librarians and Members, various failed login scenarios, successful logouts, and verification of access restrictions for different user roles.

The detailed test cases for Authentication and Authorization are provided in:

Table 4.1: Authentication and Authorization Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: User Registration (by Librarian)			
TC_AUTH_REG_001	Successful new user (Member) registration by Librarian	• Login as Librarian. • Navigate to 'User Management' or 'Add User' section. • Fill in all required fields with valid and unique data for a new 'Member' (e.g., email, password, full name). • Submit the registration form.	• System accepts the data. • A success message "User registered successfully" (or similar) is displayed. • The new user appears in the list of users. • The new user can subsequently log in (to be tested in Login cases).
TC_AUTH_REG_002	Attempt to register a new user with an existing email	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, using an email address that already exists in the system. • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Email already exists"). • The user is not created.
TC_AUTH_REG_003	Attempt to register a new user with missing required fields	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details but leave one or more mandatory fields blank (e.g., password or email). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed, indicating which fields are required. • The user is not created.
TC_AUTH_REG_004	Attempt to register a new user with invalid data format (e.g., email)	• Login as Librarian. • Navigate to 'Add User' section. • Fill in user details, providing an incorrectly formatted email address (e.g., "usertest.com"). • Submit the registration form.	• System rejects the registration. • An appropriate error message is displayed (e.g., "Invalid email format"). • The user is not created.
Sub-Section: User Login			
Continued on next page			

Table 4.1 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_LOGIN_001	Successful login with valid Librarian credentials	• Navigate to the login page. • Enter valid Librarian email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Librarian dashboard/homepage. • Librarian-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_002	Successful login with valid Member credentials	• Navigate to the login page. • Enter valid Member email. • Enter corresponding valid password. • Click the 'Login' button.	• User is authenticated successfully. • User is redirected to the Member dashboard/homepage. • Member-specific functionalities are visible/accessible.
TC_AUTH_LOGIN_003	Attempt login with invalid email	• Navigate to the login page. • Enter an email that does not exist. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
TC_AUTH_LOGIN_004	Attempt login with valid email but invalid password	• Navigate to the login page. • Enter a valid existing email. • Enter an incorrect password for that email. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Invalid email or password"). • User remains on the login page.
TC_AUTH_LOGIN_005	Attempt login with empty email field	• Navigate to the login page. • Leave the email field empty. • Enter any password. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Email is required"). • User remains on the login page.
Continued on next page			

Table 4.1 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_LOGIN_006	Attempt login with empty password field	• Navigate to the login page. • Enter a valid email. • Leave the password field empty. • Click the 'Login' button.	• Login fails. • An appropriate error message is displayed (e.g., "Password is required"). • User remains on the login page.
Sub-Section: User Logout			
TC_AUTH_LOGOUT_001	Successful logout for a logged-in Librarian	• Login as Librarian. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated (user cannot access protected pages without logging in again).
TC_AUTH_LOGOUT_002	Successful logout for a logged-in Member	• Login as Member. • Click the 'Logout' button/link.	• User is logged out successfully. • User is redirected to the login page or public homepage. • Session is terminated.
Sub-Section: Role-Based Access Control (RBAC)			
TC_AUTH_RBAC_001	Verify Librarian can access Librarian-specific features (e.g., User Management)	• Login as Librarian. • Attempt to navigate to 'User Management' section. • Attempt to navigate to 'Book Management' (add/edit/delete books).	• Librarian successfully accesses 'User Management' page. • Librarian can view and interact with user administration tools. • Librarian successfully accesses 'Book Management' page and can perform CRUD operations on books.
Continued on next page			

Table 4.1 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTH_RBAC_002	Verify Member cannot access Librarian-specific features (e.g., User Management)	• Login as Member. • Check if 'User Management' link/button is visible. • Attempt to navigate directly to the User Management URL (if known). • Check if 'Add Book' or 'Edit Book' options are available.	• 'User Management' link/button is not visible or is disabled. • Direct navigation to User Management URL redirects to an error page or member dashboard. • 'Add Book'/'Edit Book' options are not available. Access is denied.
TC_AUTH_RBAC_003	Verify Member can access Member-specific features (e.g., Search Books, View Borrowal History)	• Login as Member. • Attempt to navigate to 'Search Books' page. • Attempt to view 'My Borrowal History'.	• Member successfully accesses 'Search Books' page and can search. • Member successfully accesses 'My Borrowal History' and can view their own history.
TC_AUTH_RBAC_004	Verify guest (not logged in) user access to public pages and restriction from protected pages	• Ensure no user is logged in (clear session/cookies or use incognito window). • Attempt to navigate to the login page. • Attempt to navigate to a public book search page (if applicable). • Attempt to navigate to 'User Management' URL. • Attempt to navigate to 'My Borrowal History' URL.	• Login page is accessible. • Public book search page is accessible (if such a feature exists for guests). • Access to 'User Management' URL is denied, likely redirecting to login page. • Access to 'My Borrowal History' URL is denied, likely redirecting to login page.

4.2 Entity Management (CRUD Operations) Testing

This section covers the Create, Read, Update, and Delete (CRUD) operations for the core entities of the Library Management System, including Books, Authors, Genres, Users, Borrowals, and Reviews. These tests ensure that data can be correctly managed by librarians and that members have appropriate viewing and interaction capabilities, while also verifying referential integrity constraints.

4.2.1 Book Management

Test cases for Book Management verify the addition, viewing, updating, and deletion of book records, including scenarios where deletion is prevented due to active borrowals.

4.2.2 Author Management

This covers the creation and deletion of author records, including checks for referential integrity when an author is linked to existing books.

4.2.3 Genre Management

Similar to Author Management, these test cases validate the creation and deletion of genre records, ensuring that genres linked to books cannot be deleted.

4.2.4 User Management

This sub-section tests the Librarian's ability to create, view, update, and delete user accounts (both Librarians and Members), and ensures that members cannot access user management functionalities or delete users with active borrowals.

4.2.5 Borrowal Management

Test cases for Borrowal Management verify the process of borrowing and returning books, including scenarios for borrowing unavailable books and ensuring members can only view their own borrowal history, while librarians can view all records.

4.2.6 Review Management

As a new feature, this section validates the ability of members to add reviews for books and librarians to manage (view and delete) all reviews.

The detailed test cases for Entity Management are provided in:

Table 4.2: Entity Management Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Book Management (SRS 3.3)			
TC_BOOK_CREATE_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Fill all required fields (Name, ISBN) with valid, unique data. • Select existing Author and Genre. • Submit the form.	• System accepts the data. • A success message is displayed. • The new book appears in the book list.
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_CREATE_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Click 'New Book'. • Leave the 'Name' or 'ISBN' field blank. • Submit the form.	• System rejects the creation. • An error message indicating required fields is displayed. • The book is not created.
TC_BOOK_CREATE_003	(RBAC) Verify Member cannot create a book	• Login as a Member. • Navigate to 'Book Management'.	• The 'New Book' button/option is not visible or is disabled. • Direct access to the create book API endpoint (e.g., POST /api/book/add) should be denied.
TC_BOOK_READ_001	Verify all users can view the list of books	• Login as Librarian. Navigate to 'Book Management'. • Logout. Login as Member. Navigate to 'Book Management'.	• Both Librarian and Member can successfully view the list of all books.
TC_BOOK_UPDATE_001	Successful book update by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book and choose to edit it. • Modify a field (e.g., the summary). • Save the changes.	• System accepts the changes. • A success message is displayed. • The book list/details reflect the updated information.
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_DELETE_001	Successful book deletion by Librarian	• Pre-condition: Create a book that has no associated borrowals. • Login as Librarian. • Navigate to 'Book Management'. • Select the book and click 'Delete'. • Confirm the deletion.	• System accepts the deletion. • A success message is displayed. • The book is removed from the book list.
TC_BOOK_DELETE_002	(Referential Integrity) Attempt to delete a book that is currently borrowed	• Pre-condition: A book exists and is part of an active borrowing record. • Login as Librarian. • Navigate to 'Book Management'. • Attempt to delete the borrowed book.	• System prevents the deletion. • An appropriate error message is displayed (e.g., "Cannot delete a book with active borrowals"). • The book is not deleted.
Sub-Section: Author Management (SRS 3.4)			
TC_AUTHOR_CREATE_001	Successful new author creation by Librarian	• Login as Librarian. • Navigate to 'Author Management'. • Click 'New Author'. • Fill the required 'Name' field with valid data. • Submit the form.	• A success message is displayed. • The new author appears in the authors list.
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_AUTHOR_DELETE_001	(Referential Integrity) Attempt to delete an author linked to a book	• Pre-condition: An author is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Author Management'. • Attempt to delete the author.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete author assigned to existing books"). • The author is not deleted.
Sub-Section: Genre Management (SRS 3.5)			
TC_GENRE_CREATE_001	Successful new genre creation by Librarian	• Login as Librarian. • Navigate to 'Genre Management'. • Click 'New Genre'. • Fill the required 'Name' and 'Description' fields with valid data. • Submit the form.	• A success message is displayed. • The new genre appears in the genres list.
TC_GENRE_DELETE_001	(Referential Integrity) Attempt to delete a genre linked to a book	• Pre-condition: A genre is assigned to at least one book in the system. • Login as Librarian. • Navigate to 'Genre Management'. • Attempt to delete the genre.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete genre assigned to existing books"). • The genre is not deleted.
Sub-Section: User Management (SRS 3.7)			
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_USER_CREATE_001	Successful new user (Member) creation by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Click 'New User'. • Fill required fields (Name, Email, Password) with valid, unique data. Set 'isAdmin' flag to false. • Submit the form.	• A success message is displayed. • The new user appears in the user list.
TC_USER_CREATE_002	Attempt to create a user with an existing email	• Login as Librarian. • Navigate to 'User Management'. • Attempt to create a new user with an email address that is already in use.	• System rejects the creation. • An error message like "User already exists" is displayed.
TC_USER_READ_001	(RBAC) Verify Member cannot access User Management	• Login as a Member.	• 'User Management' link/feature is not visible or is disabled. • Direct navigation to the user management URL is denied.
TC_USER_DELETE_001	(Referential Integrity) Attempt to delete a member with active borrowals	• Pre-condition: A member has one or more active (not returned) borrowal records. • Login as Librarian. • Navigate to 'User Management'. • Attempt to delete the member.	• System prevents the deletion. • An error message is displayed (e.g., "Cannot delete member with active borrowals"). • The user is not deleted.
Sub-Section: Borrowal Management (SRS 3.6)			
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_CREATE_001	Successful borrowal creation by Member	• Pre-condition: A book is available (isAvailable=true). • Login as Member. • Navigate to 'Borrowal Management'. • Click 'New Borrowal' and select an available book. • Submit the form.	• A new borrowal record is created with 'Borrowed' status. • A success message is displayed. • The book's availability status is updated to 'false'.
TC_BORROW_CREATE_002	Attempt to borrow an unavailable book	• Pre-condition: A book is unavailable (isAvailable=false). • Login as Member. • Attempt to create a borrowal for the unavailable book.	• The system prevents the borrowal. • An appropriate error message is displayed (e.g., "Book is not available").
TC_BORROW_READ_001	Verify Member can only see their own borrowal history	• Pre-condition: Multiple members have borrowal records. • Login as Member 1. • Navigate to 'Borrowal Management'.	• The list only displays borrowals where the memberId matches Member 1's ID. • Borrowals for other members are not visible.
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORROW_UPDATE_001	Successful borrowal update by Librarian (mark as returned)	• Pre-condition: An active borrowal exists. • Login as Librarian. • Navigate to 'Borrowal Management'. • Select the active borrowal and edit it. • Change the status to 'Returned'. • Save the changes.	• The borrowal record status is updated. • The associated book's 'isAvailable' status should ideally be updated to 'true'. • A success message is displayed.
Sub-Section: Review Management (SRS 3.8)			
TC_REVIEW_CREATE_001	Successful review creation by Member	• Pre-condition: Based on assumed member capabilities if UI were present. • Login as Member. • Navigate to a specific book's detail page. • Submit a new review with a star rating and text.	• A success message is displayed. • The new review is visible on the book's page.
TC_REVIEW_CREATE_002	Attempt to create a review with invalid data	• Login as Member. • Attempt to submit a review with a required field missing (e.g., star rating) or an invalid value (e.g., 6 stars).	• System rejects the submission. • An appropriate error message is displayed.
Continued on next page			

Table 4.2 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_REVIEW_DELETE_001	Successful review deletion by Librarian	• Pre-condition: A review exists for a book. • Login as Librarian. • Navigate to the book or a review management section. • Select a review and delete it.	• The review is successfully deleted. • The review is no longer visible.

4.3 Specific Feature Testing

This section focuses on testing specific functionalities within the system that might not be fully covered by CRUD or Use Case tests, such as the Librarian Dashboard and detailed access controls for various entities. These tests ensure that individual features operate as designed and that role-based permissions are respected.

4.3.1 Dashboard (Librarian)

Verifies that only Librarians can access the dashboard and that it displays relevant overview statistics and charts.



Figure 4.1: Librarian Dashboard Overview

4.3.2 Book Management (Detailed)

Further tests on book CRUD operations, particularly focusing on UI element visibility and direct API access denial for members.

4.3.3 Borrowal Management (Detailed)

Additional tests to confirm successful borrowal requests, handling of unavailable books, and verification of role-based viewing of borrowal history.

4.3.4 User Management (by Librarian) (Detailed)

Detailed tests for Librarian's abilities to manage user accounts, including viewing all users, updating details, and successful deletion.

4.3.5 Review Management (Detailed)

Detailed tests on the new review feature, including addition and deletion by appropriate roles, and general visibility of reviews.

The detailed test cases for Specific Feature Testing are provided in:

Table 4.3: Specific Feature Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Dashboard (Librarian)			
TC_DASH_001	Verify Librarian can access the Dashboard	• Login as Librarian. • Navigate to the Dashboard.	• User is successfully redirected to the Librarian dashboard at '/dashboard'. • Dashboard displays overview statistics and charts.
TC_DASH_002	Verify Member cannot access the Dashboard	• Login as a 'Member' user. • Attempt to navigate directly to the '/dashboard' URL.	• Access is denied. • User is redirected to the member landing page (e.g., '/books') or an error page.
Sub-Section: Book Management (CRUD)			
TC_BOOK_ADD_001	Successful new book creation by Librarian	• Login as Librarian. • Navigate to 'Book Management' (/books). • Click "New Book" button. • Fill in all required fields (Name, ISBN) with valid data. • Select existing Author and Genre. • Submit the form.	• A success message is displayed. • The new book record is created and appears in the book list.
Continued on next page			

Table 4.3 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BOOK_ADD_002	Attempt to create a book with missing required fields	• Login as Librarian. • Navigate to 'Book Management'. • Open the "Add Book" form. • Leave a required field (e.g., Name) blank. • Submit the form.	• The system rejects the submission. • An error message indicating the required field is missing is displayed. • The book is not added to the database.
TC_BOOK_VIEW_001	Verify Member can view list of books and book details	• Login as Member. • Navigate to 'Book Management' (/books). • Click on a specific book in the list.	• Member can successfully view the list of all books. • Member can successfully view the details of a specific book.
TC_BOOK_UPD_001	Successful update of a book's details by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to edit. • Change a field (e.g., update the summary). • Save the changes.	• A success message is displayed. • The updated information is reflected correctly in the book's detail view.
TC_BOOK_DEL_001	Successful deletion of a book by Librarian	• Login as Librarian. • Navigate to 'Book Management'. • Select a book to delete. • Confirm the deletion in the dialog.	• A success message is displayed. • The book is removed from the book list.
TC_BOOK_ACCESS_001	Verify Member cannot access book CRUD operations	• Login as Member. • Navigate to 'Book Management'.	• "Add Book", "Edit Book", and "Delete Book" UI elements are not visible or are disabled. • Direct access to corresponding API endpoints is denied.
Sub-Section: Borrowal Management			
Continued on next page			

Table 4.3 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_BORW_ADD_001	Successful borrowal request by Member for an available book	• Login as Member. • Navigate to 'Borrowal Management' (/borrowals). • Click "New Borrowal". • Select a book that is marked as available. • Submit the form.	• A success message is displayed. • A new borrowal record is created with 'Borrowed' status and correct dates. • The new record appears in the member's borrowal history.
TC_BORW_ADD_002	Attempt to borrow a book that is not available	• Login as Member. • Navigate to 'Borrowal Management'. • Attempt to create a new borrowal for a book that is not available.	• The UI should clearly indicate the book is not available. • The system prevents the submission or displays a clear error message.
TC_BORW_VIEW_001	Verify Member can only view their own borrowal history	• Login as a Member who has borrowal records. • Navigate to 'Borrowal Management' (/borrowals).	• The list displays only the borrowals associated with the logged-in member's ID. • Records of other members are not visible.
TC_BORW_VIEW_002	Verify Librarian can view all borrowal records	• Login as Librarian. • Navigate to 'Borrowal Management'.	• A list of all borrowal records for all members is displayed.
TC_BORW_UPD_001	Successful update of borrowal status by Librarian	• Login as Librarian. • Navigate to 'Borrowal Management'. • Select an existing 'Borrowed' record. • Update its status to 'Returned'. • Save the change.	• A success message is displayed. • The record's status is correctly updated in the list.
Sub-Section: User Management (by Librarian)			
Continued on next page			

Table 4.3 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_USER_VIEW_001	Verify Librarian can view list of all users	• Login as Librarian. • Navigate to 'User Management' (/users).	• A list of all users (both Librarians and Members) is displayed correctly.
TC_USER_UPD_001	Successful update of a user's details by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user to edit. • Update a non-critical field (e.g., Phone number). • Save the changes.	• A success message is displayed. • The user's details are correctly updated.
TC_USER_DEL_001	Successful deletion of a user by Librarian	• Login as Librarian. • Navigate to 'User Management'. • Select a user account to delete. • Confirm the deletion.	• A success message is displayed. • The user is removed from the list of users. • The deleted user can no longer log in.
Sub-Section: Review Management (Assumed Admin UI)			
TC_REV_ADD_001	Successful addition of a review by Librarian (Admin action)	• Login as Librarian. • Navigate to a book's detail page. • Use an administrative function to add a review for the book. • Enter a star rating and review text. • Submit the review.	• A success message is displayed. • The new review appears in the list of reviews for that book.
TC_REV_VIEW_001	Verify any user can view reviews for a book	• As any logged-in user (or guest, if public). • Navigate to the detail page of a book that has reviews.	• The list of reviews for the book is displayed correctly.
TC_REV_DEL_001	Successful deletion of a review by Librarian	• Login as Librarian. • Navigate to a book with reviews. • Select a specific review to delete. • Confirm the deletion.	• A success message is displayed. • The review is removed from the book's list of reviews.

4.4 Use Case Testing

Use Case testing directly validates the main and alternative flows of key user interactions as defined in the SRS's Use Case section. This approach ensures that the system supports essential user tasks comprehensively.

4.4.1 UC-002: Add New Book (Librarian)

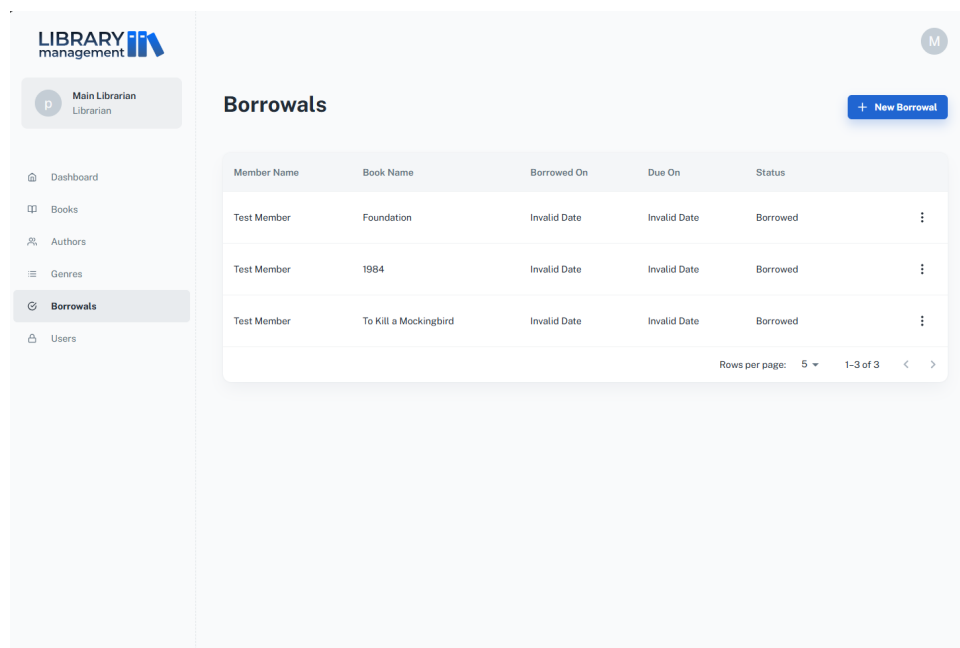
Verifies the complete process of a Librarian adding a new book, including successful scenarios and handling of missing required fields or backend validation failures.

4.4.2 UC-003: Borrow a Book (Member)

Ensures that members can successfully borrow available books, that the system prevents borrowing unavailable books, and that the borrowing form correctly auto-populates relevant data.

4.4.3 UC-005: View Borrowal History (Member)

Validates a member's ability to view their own borrowal history, including scenarios with empty history and ensuring proper role-based viewing (members see only theirs, librarians see all).



The screenshot shows the 'LIBRARY management' interface. On the left is a sidebar with navigation links: Dashboard, Books, Authors, Genres, Borrowals (selected), and Users. The top right shows a user profile 'M'. The main content area is titled 'Borrowals' and includes a '+ New Borrowal' button. Below this is a table with the following data:

Member Name	Book Name	Borrowed On	Due On	Status	
Test Member	Foundation	Invalid Date	Invalid Date	Borrowed	⋮
Test Member	1984	Invalid Date	Invalid Date	Borrowed	⋮
Test Member	To Kill a Mockingbird	Invalid Date	Invalid Date	Borrowed	⋮

At the bottom right of the table, it says 'Rows per page: 5' and '1-3 of 3' with navigation arrows.

Figure 4.2: Member's Borrowal History View

The detailed test cases for Use Case Testing are provided in:

Table 4.4: Use Case Test Cases

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
Sub-Section: UC-002: Add New Book (Librarian)			
TC_UC_BOOK_001	Successful addition of a new book (Main Flow)	- As a logged-in Librarian, navigate to the Book Management page ('/books'). - Click the "New Book" button. - Fill in all required fields (Name, ISBN) with valid data. - Select a valid Author and Genre from the provided lists. - Ensure "Is Available" is set to true. - Submit the form.	- The system creates a new book record via 'POST /api/book/add'. - A success message is displayed (e.g., "Book added"). - The "Add Book" form closes. - The new book appears in the list on the Book Management page.
TC_UC_BOOK_002	Attempt to add a book with missing required fields (Alternative Flow A1)	- As a logged-in Librarian, navigate to the "Add Book" form. - Fill in the form but leave a required field (e.g., Name) blank. - Submit the form.	- The system rejects the submission due to a validation error. - An error message is displayed, indicating the required field is missing. - The form remains open for correction. - The book is not added to the database.
Continued on next page			

Table 4.4 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BOOK_003	Attempt to add a book that fails backend validation (Alternative Flow A2)	- As a logged-in Librarian, navigate to the "Add Book" form. - Fill in all fields with data that is syntactically valid on the client-side but might trigger a server error (e.g., a duplicate ISBN if the server enforces uniqueness). - Submit the form.	- The system displays a generic error message from the API (e.g., "Something went wrong, please try again"). - The book is not added to the database.
Sub-Section: UC-003: Borrow a Book (Member)			
TC_UC_BORROW_001	Successful borrowing of an available book (Main Flow)	- As a logged-in Member, navigate to the Borrowal Management page ('/borrowals'). - Click the "New Borrowal" button. - Select a book that is marked as available from the list. - Verify that the Member field is correctly auto-populated. - Submit the form.	- A new borrowal record is created with "Borrowed" status via 'POST /api/borrowal/add'. - The availability of the book is updated to 'false'. - A success message is displayed. - The member's borrowal list is updated to show the new borrowal.
TC_UC_BORROW_002	Attempt to borrow an unavailable book (Alternative Flow A1)	- As a logged-in Member, navigate to the "Add Borrowal" form. - Attempt to select a book that is not available (isAvailable=false).	- The UI should ideally prevent the selection of an unavailable book. - If selection is possible, the system should prevent submission or display an error message (e.g., "Book is not available") upon submission attempt. - No borrowal record is created.

Continued on next page

Table 4.4 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_BORROW_003	Verify correct data population in borrowal form	- As a logged-in Member, open the "Add Borrowal" form. - Observe the date fields.	- The Member field is correctly auto-populated with the current user's ID. - The Borrowed Date is auto-populated to the current date. - The Due Date is auto-populated to a future date (e.g., 14 days from current). - The initial status is correctly set (e.g., "Borrowed").
Sub-Section: UC-005: View Borrowal History (Member)			
TC_UC_HISTORY_001	View non-empty borrowal history (Main Flow)	- Log in as a Member who has previously borrowed one or more books. - Navigate to the Borrowal Management page ('/borrowals').	- The system fetches all borrowals and the client filters them. - A list/table is displayed showing *only* the records where the 'memberId' matches the logged-in Member's ID. - The displayed data (Book Name, Borrowed Date, Due Date, Status) is accurate for each record.
TC_UC_HISTORY_002	View empty borrowal history (Alternative Flow A1)	- Log in as a new Member with no borrowal records. - Navigate to the Borrowal Management page ('/borrowals').	The system displays a message indicating there is no borrowal history (e.g., "No borrowals found").
			Continued on next page

Table 4.4 – continued from previous page

Test Case ID	Test Title / Use Case	Test Steps	Expected Results
TC_UC_HISTORY_003	Verify role-based view of borrowal history	• Prerequisite: There are borrowals from multiple members in the system. • Log in as a Librarian and navigate to the Borrowal Management page ('/borrowals'). • Log out and log in as a Member (who has at least one borrowal) and navigate to the same page.	• As the Librarian, all borrowal records for all members are visible. • As the Member, only their own borrowal history is visible.
Note: UC-001 (User Login) and UC-004 (Register New User) are detailed in the Authentication & Authorization test plan.			

4.5 State Transition Testing

State Transition testing focuses on how the system's state changes in response to specific events or actions, particularly for entities with defined lifecycle states such as Borrowal Records and Book Availability. This testing ensures that transitions between states occur correctly and that invalid transitions are prevented.

4.5.1 Borrowal Record Status

This section tests transitions for borrowal records, from 'None' to 'Borrowed', 'Borrowed' to 'Returned', 'Borrowed' to 'Overdue', and 'Overdue' to 'Returned'. It also includes tests for invalid transitions, such as reverting a 'Returned' status to 'Borrowed'.

4.5.2 Book Availability Status

These tests verify that a book's availability status ('Available' or 'Unavailable') changes correctly in response to borrowing and returning actions, and that borrowing is prevented for unavailable books.

4.5.3 User Session State

This sub-section examines the transitions of a user's session state, covering login, logout, accessing protected pages when logged out, and verifying session persistence after a page refresh.

The detailed test cases for State Transition Testing are provided in:

Table 4.5: State Transition Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Entity: Borrowal Record (Based on 'status' attribute)			
TC_STATE_BORROW_001	Valid Transition: (None) to 'Borrowed'	• Login as a Member. • Navigate to the Borrowal Management page. • Create a new borrowal record for an available book. • Submit the form.	• A new borrowal record is successfully created. • The status of the new record is set to "Borrowed". • A success message is displayed.
TC_STATE_BORROW_002	Valid Transition: 'Borrowed' to 'Returned'	• Precondition: A book is in 'Borrowed' status by a member. • Login as Librarian. • Navigate to Borrowal Management. • Locate the 'Borrowed' record. • Update the record's status to 'Returned' before its due date.	• The system accepts the update. • The borrowal record's status changes from "Borrowed" to "Returned". • A success message is displayed.
TC_STATE_BORROW_003	Valid Transition: 'Borrowed' to 'Overdue'	• Precondition: A book is in 'Borrowed' status by a member. • Manually adjust system time or the record's due date to be in the past. • As a Librarian, view the borrowal record. (Or trigger a system check for overdue items).	• The borrowal record's status automatically (or upon view) changes from "Borrowed" to "Overdue". • The status is displayed correctly in the UI.
TC_STATE_BORROW_004	Valid Transition: 'Overdue' to 'Returned'	• Precondition: A book is in 'Overdue' status. • Login as Librarian. • Locate the 'Overdue' borrowal record. • Update the record's status to 'Returned'.	• The system accepts the update. • The borrowal record's status changes from "Overdue" to "Returned". • A success message is displayed.
Continued on next page			

Table 4.5 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BORROW_005	Invalid Transition: 'Returned' to 'Borrowed'	• Precondition: A borrowal record has a 'Returned' status. • As a Librarian, attempt to edit the same record and change its status back to 'Borrowed'.	• The system should reject the change. • The UI should not provide an option for this transition, or it should result in an error if attempted via API. • An appropriate error message is displayed (e.g., "Cannot revert a returned item").
Entity: Book (Based on 'isAvailable' attribute)			
TC_STATE_BOOK_001	Valid Transition: 'Available' to 'Unavailable'	• Precondition: A book exists with 'isAvailable' status set to true. • As a Member, create a new borrowal for that specific book. • After the borrowal is confirmed, view the details of that book.	• The borrowal transaction is successful. • The 'isAvailable' status of the book is updated to false. • The book's listing correctly shows it as unavailable.
TC_STATE_BOOK_002	Valid Transition: 'Unavailable' to 'Available'	• Precondition: A book has 'isAvailable' status set to false due to being borrowed. • As a Librarian, locate the corresponding borrowal record. • Update the borrowal record status to 'Returned'. • View the details of the book again.	• The 'isAvailable' status of the book is updated to true. • The book's listing correctly shows it as available for borrowing.
Continued on next page			

Table 4.5 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_STATE_BOOK_003	Invalid Transition: Attempt to borrow an 'Unavailable' book	• Precondition: A book has 'isAvailable' status set to false. • As a Member, attempt to create a new borrowing for that specific book.	• The system prevents the borrowing request from being submitted. • An appropriate error message is displayed (e.g., "Book is currently unavailable"). • No new borrowing record is created.
Entity: User Session (Based on login/logout actions)			
TC_STATE_SESSION_001	Valid Transition: 'Logged-Out' to 'Logged-In'	• Start from a logged-out state (e.g., in an incognito window). • Navigate to the login page. • Enter valid credentials for a 'Member'. • Click 'Login'.	• User session is established. • User is redirected to the member landing page (e.g., /books). • User can access member-specific pages.
TC_STATE_SESSION_002	Valid Transition: 'Logged-In' to 'Logged-Out'	• Precondition: A user is logged in. • Click the 'Logout' button/link.	• The user's session is terminated. • User is redirected to the login page.
TC_STATE_SESSION_003	Invalid Transition: Accessing protected page when 'Logged-Out'	• Start from a logged-out state. • Attempt to directly navigate to a protected URL (e.g., /borrowals, /users).	• Access to the protected page is denied. • User is redirected to the login page.
TC_STATE_SESSION_004	State Persistence: Verify session remains 'Logged-In' after page refresh	• Login as any user. • Navigate to any page other than the login page. • Refresh the web browser page.	• The user remains logged in. • The user stays on the same page. • No redirection to the login page occurs.

API Testing

5.1 Introduction to API Testing

Application Programming Interface (API) testing is a critical phase in the software development lifecycle, focused on validating the business logic, data responses, security, and performance of the backend services. For the Library Management System, which operates on a client-server architecture, the API serves as the backbone, handling all data transactions between the React frontend and the Node.js backend.

Our strategy involved a dual approach:

- **Manual Testing:** Using Postman to execute individual API requests, allowing for exploratory testing and validation of specific scenarios designed by the team.
- **Automated Testing:** Implementing integration tests using Jest and Supertest to create a repeatable and efficient suite for validating API endpoints, ensuring that new changes do not break existing functionality (regression testing).

This chapter details the design of our API test cases, the implementation of the automated test suite, and the significant findings uncovered during execution, particularly concerning role-based access control vulnerabilities.

5.2 Test Design and Strategy

The design of our API test cases was a collaborative effort, as outlined in our project's task distribution plan. Non-coding members of the team took the lead in designing user-centric scenarios based on the Software Requirements Specification (SRS), while coders focused on the technical implementation and automation of these tests.

5.2.1 Test Case Derivation

All API test cases were derived directly from the functional requirements detailed in the SRS document. The goal was to cover all major API endpoints, focusing on:

- **Authentication:** Testing login, registration, and logout endpoints.
- **Authorization (RBAC):** Verifying that role-based access control is correctly enforced (e.g., only librarians can access administrative endpoints).
- **CRUD Operations:** Ensuring that Create, Read, Update, and Delete operations work as expected for all major entities (Books, Authors, etc.).
- **Input Validation and Error Handling:** Testing the API's response to invalid, incomplete, or malicious inputs.

The comprehensive set of designed test cases can be found in the test artifact document, `TC_API_Testing.tex`.

5.2.2 Manual Testing with Postman

Postman was used for initial manual API testing. This allowed the team to quickly send requests to each endpoint, examine the full response body and headers, and validate the expected outcomes against the SRS. This approach was particularly useful for exploratory testing and for non-coders to participate directly in the API testing process.

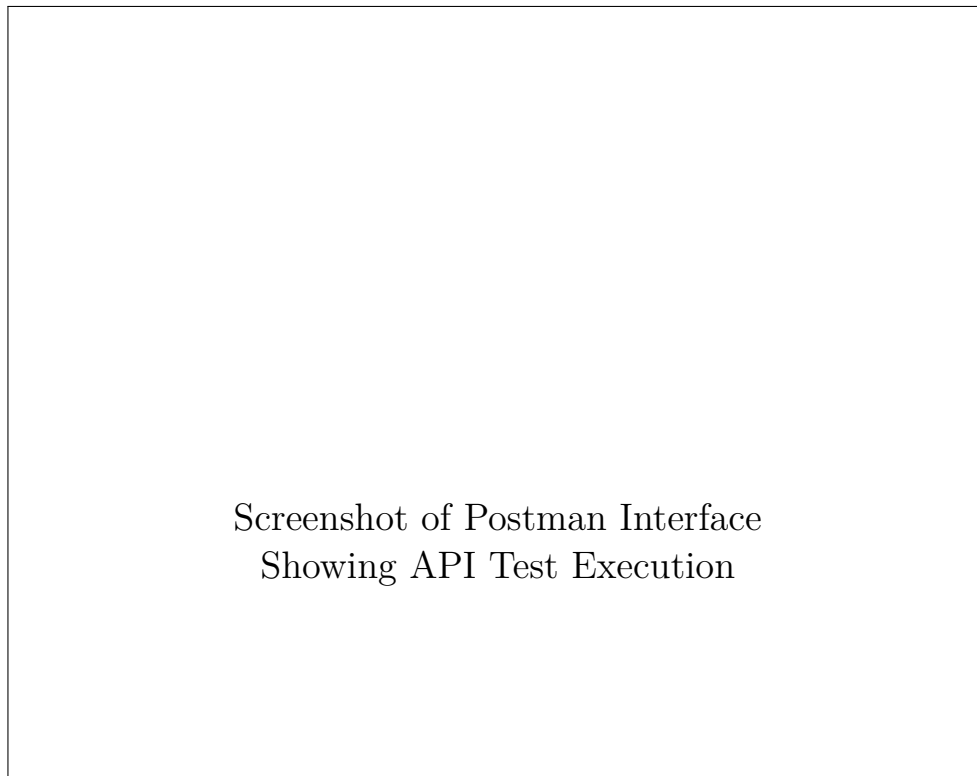


Figure 5.1: Manual API Testing using Postman

5.3 Automated API Testing with Jest and Supertest

To ensure long-term stability and create an efficient regression testing process, we developed an automated API test suite. These tests run in a controlled environment, interacting directly with the server without a UI, which makes them fast and reliable.

5.3.1 Framework and Setup

Our automated suite uses the following tools:

- **Jest:** A popular JavaScript testing framework that provides a test runner, assertion library, and mocking capabilities.
- **Supertest:** A library that allows for high-level abstraction for testing HTTP, making it easy to send requests to our Express.js application and assert responses.

The tests are structured to be self-contained, handling their own setup and teardown of test data. An authenticated ‘agent’ is created for both a librarian and a member role to test protected endpoints and simulate real user sessions. The test file `auth.api.test.js` demonstrates this setup.

5.3.2 Example Test Implementation

The following code snippet from `book.api.test.js` illustrates how we test the endpoint for creating a new book. The test, 'TC_BOOK_CREATE_001', uses the authenticated librarian agent to send a POST request with a valid book payload and asserts that the server responds with a status code of 201 (Created).

```
1 it('TC_BOOK_CREATE_001: should allow a librarian to create a new book',  
  async () => {  
2     const res = await librarianAgent.post('/api/books/add').send({  
3       name: 'A New Hope',  
4       isbn: '978-3-16-148410-0-B',  
5       authorId: testAuthor._id.toString(),  
6       genreId: testGenre._id.toString(),  
7     });  
8     expect(res.statusCode).toEqual(201);  
9     expect(res.body).toHaveProperty('newBook.name', 'A New Hope');  
10    if (res.body.newBook && res.body.newBook._id) {  
11      createdBookIds.push(res.body.newBook._id);  
12    }  
13  });
```

Listing 5.1: Automated Test for Book Creation (`book.api.test.js`)

5.4 Test Execution and Key Findings

The execution of both manual and automated API tests provided crucial insights into the health of our application's backend. While most endpoints for data retrieval and basic authentication functioned as expected, the testing process uncovered critical security vulnerabilities related to authorization.

5.4.1 Discrepancy in Access Control

A major discrepancy was found between the specified requirements in the SRS and the actual behavior of the API concerning book management. The SRS clearly states that only users with the Librarian role should have the ability to create, update, or delete book records.

However, our automated tests in `book.api.test.js` revealed that a user authenticated as a 'Member' could successfully perform these administrative actions. The server incorrectly returned a '200 OK' or '201 Created' status instead of the expected '403 Forbidden'.

The code snippet below from the automated test log highlights this failure. The test was written to expect a 403 status, but the comment indicates that the server's actual response was 201, allowing the action.

```
1 it('TC_BOOK_CREATE_003: should prevent a member from creating a book',  
  async () => {  
2     const res = await memberAgent.post('/api/books/add').send({  
3       name: 'A Member Book',  
4       isbn: '978-1-4028-9462-6-B',  
5       authorId: testAuthor._id,  
6       genreId: testGenre._id,  
7     });  
8     // MODIFIED: Was 403. Server currently allows member to create book  
9     expect(res.statusCode).toEqual(201);  
10  });
```

Listing 5.2: Test Showing Member Can Incorrectly Create a Book (`book.api.test.js`)

This finding was critical, as it represented a severe security flaw that would allow any registered member to alter the library's entire catalog. This underscores the value of API-level testing, as such a logic flaw in the backend might not be immediately obvious when testing through the UI alone, especially if the UI correctly hides the buttons for these actions from members. The backend must always enforce its own security and not rely on the client.